Animixel

0.0.1

Generated by Doxygen 1.18.0

1 Data Structure Index 1

1.1 Data Structures . . . 1

2 File Index 3

2.1 File List . . . 3

3 Data Structure Documentation 5

3.1 Custom_Element_Data::[union].__unnamed0__ Union Reference . . . 5

3.2 Button_State_Manager Struct Reference . . . 5

3.2.1 Detailed Description . . . 5

3.3 Canvas Struct Reference . . . 6

3.3.1 Detailed Description . . . 6

3.4 Custom_Element_Data Struct Reference . . . 6

3.4.1 Detailed Description . . . 7

3.5 Input_Handler Struct Reference . . . 7

3.5.1 Detailed Description . . . 7

4 File Documentation 9

4.1 src/canvas.h File Reference . . . 9

4.1.1 Detailed Description . . . 10

4.1.2 Function Documentation . . . 10

4.1.2.1 canvas_begin_shape() . . . 10

4.1.2.2 canvas_bresenham() . . . 10

4.1.2.3 canvas_draw() . . . 11

4.1.2.4 canvas_draw_empty_circle() . . . 11

4.1.2.5 canvas_draw_empty_rect() . . . 12

4.1.2.6 canvas_draw_filled_circle() . . . 12

4.1.2.7 canvas_draw_filled_rect() . . . 13

4.1.2.8 canvas_draw_line() . . . 13

4.1.2.9 canvas_dump_set() . . . 14

4.1.2.10 canvas_end_shape() . . . 14

4.1.2.11 canvas_free() . . . 14

4.1.2.12 canvas_init() . . . 14

4.1.2.13 canvas_print() . . . 15

4.1.2.14 canvas_set_pixel() . . . 15

4.2 canvas.h . . . 16

4.3 src/input.h File Reference . . . 16

4.3.1 Detailed Description . . . 17

4.3.2 Enumeration Type Documentation . . . 17

4.3.2.1 Button_Type . . . 17

4.3.2.2 keystate . . . 18

4.3.3 Function Documentation . . . 18

4.3.3.1 button_state_manager_init() . . . 18

4.3.3.2 cursor_in_bounds() . . . 19
4.3.3.3 input_handler_init() . . . 19
4.3.3.4 process_input() . . . 19
4.3.3.5 update_key_state() . . . 20
4.3.3.6 update_sbp() . . . 20
4.4 input.h . . . 20
4.5 src/math_utils.h File Reference . . . 21
4.5.1 Detailed Description . . . 22
4.6 math_utils.h . . . 22
4.7 src/ui_core.h File Reference . . . 22
4.7.1 Detailed Description . . . 23
4.7.2 Function Documentation . . . 23
4.7.2.1 clay_init() . . . 23
4.7.2.2 clay_render() . . . 24
4.7.2.3 clay_update_dimensions() . . . 24
4.7.2.4 create_static_button() . . . 24
4.7.2.5 editor_layout() . . . 25
4.8 ui_core.h . . . 25

Index **27**

Chapter 1

Data Structure Index

1.1 Data Structures

Here are the data structures with brief descriptions:

Custom_Element_Data::[union].__unnamed0__ 5
Button_State_Manager
Struct to manage the state of all the buttons similar to Input_Handler 5
Canvas
Struct that describes a drawing canvas 6
Custom_Element_Data
Struct used to hold the data of a Clay custom element as a tagged union 6
Input_Handler
Struct that acts as input handling interface 7

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

src/canvas.h
: Contains the definition for a Canvas and its associated functions 9

src/input.h
: Contains structs/functions for handling user input 16

src/math_utils.h
: Contains the definition various mathematical helper functions/values 21

src/ui_core.h
: Contains structs/functions for UI using Clay 22

Chapter 3

Data Structure Documentation

3.1 Custom_Element_Data::[union].__unnamed0__ Union Reference

Data Fields

- Canvas * **canvas**

  Tagged union pointer element for a canvas.

The documentation for this union was generated from the following files:

3.2 Button_State_Manager Struct Reference

Struct to manage the state of all the buttons similar to Input_Handler.

`#include <input.h>`

Data Fields

- Button_Type **stb**

  Selected toolbar button.
- Button_Type **smb**

  Selected menu button.
- Clay_String **sbp**

  Shape button pressed.

3.2.1 Detailed Description

Struct to manage the state of all the buttons similar to Input_Handler.

The documentation for this struct was generated from the following file:

- src/input.h

3.3 Canvas Struct Reference

Struct that describes a drawing canvas.

`#include <canvas.h>`

Data Fields

- uint32_t * **pixels**

  Array holding the pixel data.
- uint32_t * **prev_pixels**

  Pixel data from before shape drawing started.
- BOB_Pixelbuffer_Handle **pb**

  Reference handle to the pixelbuffer that actually renders the canvas data to the screen.
- size_t **width**

  Width of the canvas.
- size_t **height**

  Height of the canvas.
- size_t **scale**

  Scale factor of the canvas. Used when rendering.
- uint32_t **origin_x**

  Top left x coord.
- uint32_t **origin_y**

  Top left y coord.
- uint32_t **shape_x**

  X coord from where a shape drawing started.
- uint32_t **shape_y**

  Y coord from where a shape drawing started.

3.3.1 Detailed Description

Struct that describes a drawing canvas.

The documentation for this struct was generated from the following file:

- src/canvas.h

3.4 Custom_Element_Data Struct Reference

Struct used to hold the data of a Clay custom element as a tagged union.

`#include <ui_core.h>`

Data Structures

- union [union].__unnamed0__

Data Fields

- Custom_Element_Type **type**
  Tag that indicates the type of the custom element data.
- union

3.4.1 Detailed Description

Struct used to hold the data of a Clay custom element as a tagged union.

All of the union elements are pointers to maintain a consistent struct size

The documentation for this struct was generated from the following file:

- src/ui_core.h

3.5 Input_Handler Struct Reference

Struct that acts as input handling interface.

`#include <input.h>`

Data Fields

- keystate **key_status** [NUM_KEYS]
  Map that holds the state of all the current keys we care about.
- double **mouse_x**
  Mouse x position in the current frame.
- double **mouse_y**
  Mouse y position in the current frame.
- double **prev_x**
  Mouse x position in the previous frame.
- double **prev_y**
  Mouse y position in the previous frame.
- double **scroll_x**
  Mouse horizontal scroll in the current frame.
- double **scroll_y**
  Mouse vertical scroll in the current frame.

3.5.1 Detailed Description

Struct that acts as input handling interface.

The documentation for this struct was generated from the following file:

- src/input.h

Chapter 4

File Documentation

4.1 src/canvas.h File Reference

Contains the definition for a Canvas and its associated functions.

```
#include <stdint.h>
#include <stddef.h>
#include <bob.h>
```

Data Structures

- struct Canvas

  Struct that describes a drawing canvas.

Functions

- void canvas_init (Canvas *c, size_t width, size_t height, size_t scale, uint32_t default_colour, BOB_↩ Renderer_Handle r)

  Initialises a Canvas.
- void canvas_free (Canvas *c)

  Frees all memory used by member variables of a Canvas outside of the struct itself, but does not destroy the memory region occupied by the struct.
- void canvas_draw (Canvas *c, BOB_Renderer_Handle renderer)

  Draws a canvas to the screen.
- void canvas_set_pixel (Canvas *c, int32_t x, int32_t y, uint32_t colour)

  Sets a pixel in the Canvas.
- void canvas_bresenham (Canvas *c, int32_t sx, int32_t sy, int32_t ex, int32_t ey, uint32_t colour)

  Uses Bresenham's line algorithm to draw a line between two points on a Canvas.
- void canvas_begin_shape (Canvas *c, int32_t sx, int32_t sy)

  Function to be called before shape drawing starts.
- void canvas_end_shape (Canvas *c)

  Ends shape drawing by resetting the Canvas' shape_x/y member variables to the sentinel value of UINT32_MAX.
- void canvas_draw_line (Canvas *c, int32_t sx, int32_t sy, int32_t px, int32_t py)

  Draws a line by running two passes of Bresenham's algorithm.

- void canvas_draw_filled_rect (Canvas *c, int32_t ex, int32_t ey, int32_t px, int32_t py)

  Draws a filled rect in two passes of the Canvas grid.
- void canvas_draw_empty_rect (Canvas *c, int32_t ex, int32_t ey, int32_t px, int32_t py)

  Draws a empty rect in two passes of the Canvas grid.
- void canvas_draw_filled_circle (Canvas *c, int32_t ex, int32_t ey, int32_t px, int32_t py)

  Draws a filled circle in two passes of the Canvas grid.
- void canvas_draw_empty_circle (Canvas *c, int32_t ex, int32_t ey, int32_t px, int32_t py)

  Draws an empty circle in two passes of the Canvas grid using the Midpoint Circle Algorithm.
- void canvas_print (Canvas *c)

  Prints the pixels array of the Canvas to the terminal.
- void canvas_dump_set (Canvas *c)

  Prints the set pixels array of the Canvas to the terminal.

4.1.1 Detailed Description

Contains the definition for a Canvas and its associated functions.

4.1.2 Function Documentation

4.1.2.1 canvas_begin_shape()

```
void canvas_begin_shape (
            Canvas * c,
            int32_t sx,
            int32_t sy)
```

Function to be called before shape drawing starts.

Sets the start position of the shape (where the user first clicked) in the Canvas

Parameters

c	a pointer to the Canvas where the shape drawing started
sx	the x coordinate of the shape start position
sy	the y coordinate of the shape start position

4.1.2.2 canvas_bresenham()

```
void canvas_bresenham (
            Canvas * c,
            int32_t sx,
            int32_t sy,
            int32_t ex,
            int32_t ey,
            uint32_t colour)
```

Uses Bresenham's line algorithm to draw a line between two points on a Canvas.

Parameters

c	a pointer to the Canvas where the line should be drawn
sx	the starting x coord of the line
sy	the starting y coord of the line
ex	the ending x coord of the line
ey	the ending y coord of the line
colour	the colour the line whould be drawn in

4.1.2.3 canvas_draw()

```
void canvas_draw (
            Canvas * c,
            BOB_Renderer_Handle renderer)
```

Draws a canvas to the screen.

Updates the Canvas' pixelbuffer with the new pixel values from the pixels array. Determines the size of the destination quad using the width, height and scale member variables

Parameters

c	a pointer to the Canvas to be drawn
r	a handle to the renderer that was used to create the Canvas' pixelbuffer

4.1.2.4 canvas_draw_empty_circle()

```
void canvas_draw_empty_circle (
            Canvas * c,
            int32_t ex,
            int32_t ey,
            int32_t px,
            int32_t py)
```

Draws an empty circle in two passes of the Canvas grid using the Midpoint Circle Algorithm.

The first clears the circle with radius = sqrt(pow(px - start_x, 2), pow(py - start_y, 2)) to the colours stored in the prev_pixels snapshot The second draws a new circle with radius = sqrt(pow(sx - start_x, 2), pow(sy - start_y, 2))

Parameters

c	a pointer to the canvas where the circle should be drawn
sx	the x coordinate of the mouse pointer this frame
sy	the y coordinate of the mouse pointer this frame
px	the x coordinate of the mouse pointer last frame
py	the y coordinate of the mouse pointer last frame

4.1.2.5 canvas_draw_empty_rect()

```
void canvas_draw_empty_rect (
            Canvas * c,
            int32_t ex,
            int32_t ey,
            int32_t px,
            int32_t py)
```

Draws a empty rect in two passes of the Canvas grid.

The first clears the rect drawn between (start_x, start_y) and (px, py) to the colours stored in the prev_pixels snapshot. The second draws a new rect between (start_x, start_y) and (sx, sy)

Parameters

c	a pointer to the canvas where the rect should be drawn
sx	the x coordinate of the rect endpoint this frame
sy	the y coordinate of the rect endpoint this frame
px	the x coordinate of the rect endpoint last frame
py	the y coordinate of the rect endpoint last frame

4.1.2.6 canvas_draw_filled_circle()

```
void canvas_draw_filled_circle (
            Canvas * c,
            int32_t ex,
            int32_t ey,
            int32_t px,
            int32_t py)
```

Draws a filled circle in two passes of the Canvas grid.

The first clears the circle with radius = sqrt(pow(px - start_x, 2), pow(py - start_y, 2)) to the colours stored in the prev_pixels snapshot The second draws a new circle with radius = sqrt(pow(sx - start_x, 2), pow(sy - start_y, 2))

Parameters

c	a pointer to the canvas where the circle should be drawn
sx	the x coordinate of the mouse pointer this frame
sy	the y coordinate of the mouse pointer this frame
px	the x coordinate of the mouse pointer last frame
py	the y coordinate of the mouse pointer last frame

4.1.2.7 canvas_draw_filled_rect()

```
void canvas_draw_filled_rect (
            Canvas * c,
            int32_t ex,
            int32_t ey,
            int32_t px,
            int32_t py)
```

Draws a filled rect in two passes of the Canvas grid.

The first clears the rect drawn between (start_x, start_y) and (px, py) to the colours stored in the prev_pixels snapshot. The second draws a new rect between (start_x, start_y) and (sx, sy)

Parameters

c	a pointer to the canvas where the rect should be drawn
sx	the x coordinate of the rect endpoint this frame
sy	the y coordinate of the rect endpoint this frame
px	the x coordinate of the rect endpoint last frame
py	the y coordinate of the rect endpoint last frame

4.1.2.8 canvas_draw_line()

```
void canvas_draw_line (
            Canvas * c,
            int32_t sx,
            int32_t sy,
            int32_t px,
            int32_t py)
```

Draws a line by running two passes of Bresenham's algorithm.

The first clears the line drawn between (start_x, start_y) and (px, py) to the colours stored in the prev_pixels snapshot The second draws a new line between (start_x, start_y) and (sx, sy)

Parameters

c	a pointer to the canvas where the line should be drawn
sx	the x coordinate of the line endpoint this frame
sy	the y coordinate of the line endpoint this frame
px	the x coordinate of the line endpoint last frame
py	the y coordinate of the line endpoint last frame

4.1.2.9 canvas_dump_set()

```
void canvas_dump_set (
            Canvas * c)
```

Prints the set pixels array of the Canvas to the terminal.

Used for debugging and should not be used in production code. Preffered over canvas_print for large sizes where printing the full Canvas would be unreadable

Parameters

c	a pointer to the Canvas whose set pixels should be printed

4.1.2.10 canvas_end_shape()

```
void canvas_end_shape (
            Canvas * c)
```

Ends shape drawing by resetting the Canvas' shape_x/y member variables to the sentinel value of UINT32_MAX.

Parameters

c	a pointer to Canvas where the shape drawing finished

4.1.2.11 canvas_free()

```
void canvas_free (
            Canvas * c)
```

Frees all memory used by member variables of a Canvas outside of the struct itself, but does not destroy the memory region occupied by the struct.

Parameters

c	a pointer to the Canvas whole members must be freed

4.1.2.12 canvas_init()

```
void canvas_init (
            Canvas * c,
            size_t width,
            size_t height,
            size_t scale,
            uint32_t default_colour,
            BOB_Renderer_Handle r)
```

Initialises a Canvas.

Does not allocate memory for the struct itself (this must be done beforehand using either a local struct variable or a heap allocation), but will allocate memory for the pixels buffer and will create the PixelBuffer via a call to BOB. Sets origin_x/y to default values of 0 and shape_x/y to sentinel values of UINT32_MAX

Parameters

c	a pointer to a Canvas to be initialised
width	the desired width of the Canvas
height	the desired height of the Canvas
scale	the desired scale factor of the Canvas
default_colour	the background colour the Canvas should default to when first drawn
r	the handle of the BOB_Renderer the Canvas should use to generate its pixelbuffer

4.1.2.13 canvas_print()

```
void canvas_print (
            Canvas * c)
```

Prints the pixels array of the Canvas to the terminal.

Used for debugging and should not be used in production code

Parameters

c	a pointer to the Canvas whose pixels should be printed

4.1.2.14 canvas_set_pixel()

```
void canvas_set_pixel (
            Canvas * c,
            int32_t x,
            int32_t y,
            uint32_t colour)
```

Sets a pixel in the Canvas.

Only updates the pixels array, not the pixelbuffer

Parameters

c	a pointer to the Canvas where the pixel should be set
x	the x coordinate of the pixel
y	the y coordinate of the pixel
colour	the new colour the pixel should be set to

4.2 canvas.h

Go to the documentation of this file.

```
00001
00005
00006 #ifndef CANVAS_H
00007 #define CANVAS_H
00008
00009 #include <stdint.h>
00010 #include <stddef.h>
00011 #include <bob.h>
00012
00014 typedef struct {
00015     uint32_t *pixels;
00016     uint32_t *prev_pixels;
00017     BOB_Pixelbuffer_Handle pb;
00018     size_t width;
00019     size_t height;
00020     size_t scale;
00021     uint32_t origin_x;
00022     uint32_t origin_y;
00023     uint32_t shape_x;
00024     uint32_t shape_y;
00025 } Canvas;
00026
00039 void canvas_init(Canvas *c, size_t width, size_t height, size_t scale, uint32_t default_colour,
      BOB_Renderer_Handle r);
00045 void canvas_free(Canvas *c);
00052 void canvas_draw(Canvas *c, BOB_Renderer_Handle renderer);
00060 void canvas_set_pixel(Canvas *c, int32_t x, int32_t y, uint32_t colour);
00070 void canvas_bresenham(Canvas *c, int32_t sx, int32_t sy, int32_t ex, int32_t ey, uint32_t colour);
00078 void canvas_begin_shape(Canvas *c, int32_t sx, int32_t sy);
00084 void canvas_end_shape(Canvas *c);
00096 void canvas_draw_line(Canvas *c, int32_t sx, int32_t sy, int32_t px, int32_t py);
00108 void canvas_draw_filled_rect(Canvas *c, int32_t ex, int32_t ey, int32_t px, int32_t py);
00120 void canvas_draw_empty_rect(Canvas *c, int32_t ex, int32_t ey, int32_t px, int32_t py);
00132 void canvas_draw_filled_circle(Canvas *c, int32_t ex, int32_t ey, int32_t px, int32_t py);
00144 void canvas_draw_empty_circle(Canvas *c, int32_t ex, int32_t ey, int32_t px, int32_t py);
00145
00151 void canvas_print(Canvas *c);
00158 void canvas_dump_set(Canvas *c);
00159
00160 #endif
```

4.3 src/input.h File Reference

Contains structs/functions for handling user input.

```
#include <clay.h>
#include "canvas.h"
```

Data Structures

- struct Input_Handler

  Struct that acts as input handling interface.
- struct Button_State_Manager

  Struct to manage the state of all the buttons similar to Input_Handler.

Enumerations

- enum keystate { KEY_JUST_PRESSED , KEY_PRESSED , KEY_RELEASED , KEY_JUST_RELEASED }

  Enum to hold the state of the keys.
- enum keys {
  KEY_0 , **KEY_1** , **KEY_2** , **KEY_3** ,
  KEY_4 , **KEY_5** , **KEY_6** , **KEY_7** ,
  KEY_8 , **KEY_9** , **KEY_A** , **KEY_B** ,
  KEY_C , **KEY_D** , **KEY_E** , **KEY_F** ,
  KEY_G , **KEY_H** , **KEY_I** , **KEY_J** ,
  KEY_K , **KEY_L** , **KEY_M** , **KEY_N** ,
  KEY_O , **KEY_P** , **KEY_Q** , **KEY_R** ,
  KEY_S , **KEY_T** , **KEY_U** , **KEY_V** ,
  KEY_W , **KEY_X** , **KEY_Y** , **KEY_Z** ,
  KEY_MINUS , **KEY_PLUS** , **MOUSE_BUTTON_LEFT** , **MOUSE_BUTTON_RIGHT** ,
  MOUSE_BUTTON_MIDDLE , **NUM_KEYS** }

  Enum that holds all of the keys we currently want to work with.
- enum Button_Type {
  BUTTON_DRAW , BUTTON_ERASE , BUTTON_SHAPE , BUTTON_FILE ,
  BUTTON_LINE , BUTTON_EMPTY_RECT , BUTTON_FILLED_RECT , BUTTON_EMPTY_CIRCLE ,
  BUTTON_FILLED_CIRCLE , BUTTON_NONE }

  Enum to hold the different buttons.

Functions

- void input_handler_init (Input_Handler *ih)

  Initialises an Input_Handler.
- void update_key_state (Input_Handler *ih)

  Updates the key state after user input processing.
- int cursor_in_bounds (Input_Handler *ih, float min_x, float min_y, float max_x, float max_y)

  Simple checker that returns if the cursor is in the bounds [min_x, max_x), [min_y, max_y).
- void button_state_manager_init (Button_State_Manager *bs)

  Initialises a Button_State_Manager.
- void process_input (Input_Handler *ih, Button_State_Manager *bs_manager, Canvas *c)

  Processes user input and updates button state.
- void update_sbp (Button_State_Manager *bs_manager, Button_Type t)

  Helper function to update the string displayed in the Shape button Called when a button/button hotkey is pressed.

4.3.1 Detailed Description

Contains structs/functions for handling user input.

4.3.2 Enumeration Type Documentation

4.3.2.1 Button_Type

```
enum Button_Type
```

Enum to hold the different buttons.

Enumerator

BUTTON_DRAW	Button to draw freehand.
BUTTON_ERASE	Button to erase.
BUTTON_SHAPE	Button to select a shape to draw with.
BUTTON_FILE	Button to open the file menu.
BUTTON_LINE	Button to start drawing a line.
BUTTON_EMPTY_RECT	Button to start drawing an empty rect.
BUTTON_FILLED_RECT	Button to start drawing a filled rect.
BUTTON_EMPTY_CIRCLE	Button to start drawing an empty circle.
BUTTON_FILLED_CIRCLE	Button to start drawing a filled circle.
BUTTON_NONE	Sentinel value to represent no button selected.

4.3.2.2 keystate

```
enum keystate
```

Enum to hold the state of the keys.

Enumerator

KEY_JUST_PRESSED	Was pressed this frame.
KEY_PRESSED	Held on from the previous frame.
KEY_RELEASED	Not pressed.
KEY_JUST_RELEASED	Released the previous frame.

4.3.3 Function Documentation

4.3.3.1 button_state_manager_init()

```
void button_state_manager_init (
            Button_State_Manager * bs)
```

Initialises a Button_State_Manager.

Sets button values to BUTTON_NONE and sbp to "S"

Parameters

bs	a pointer to the Button_State_Manager to initialise

4.3.3.2 cursor_in_bounds()

```
int cursor_in_bounds (
            Input_Handler * ih,
            float min_x,
            float min_y,
            float max_x,
            float max_y)
```

Simple checker that returns if the cursor is in the bounds [min_x, max_x), [min_y, max_y).

Parameters

ih	a pointer to the Input_Handler whose mouse position we are checking
min_x	the minimum x value the cursor can hold
min_y	the minimum y value the cursor can hold
max_x	the maximum x value the cursor can hold
max_y	the maximum y value the cursor can hold

Returns

1 if the cursor is in the bounds of the window, 0 otherwise

4.3.3.3 input_handler_init()

```
void input_handler_init (
            Input_Handler * ih)
```

Initialises an Input_Handler.

Sets the initial cursor position to (0, 0) and all of the keys to KEY_RELEASED Does not allocate memory for the Input_Handler, this must be done before the function was called

Parameters

ih	a pointer to the Input_Handler to be initialised

4.3.3.4 process_input()

```
void process_input (
            Input_Handler * ih,
            Button_State_Manager * bs_manager,
            Canvas * c)
```

Processes user input and updates button state.

Parameters

ih	a pointer to the Input_Handler holding the input state
bs_manager	a pointer to the Button_State_Manager holding the button state
c	a pointer to a canvas that could be affected by user input

4.3.3.5 update_key_state()

```
void update_key_state (
            Input_Handler * ih)
```

Updates the key state after user input processing.

Parameters

ih	a pointer to the Input_Handler whose key map needs updating

4.3.3.6 update_sbp()

```
void update_sbp (
            Button_State_Manager * bs_manager,
            Button_Type t)
```

Helper function to update the string displayed in the Shape button Called when a button/button hotkey is pressed.

Parameters

bs_manager	a pointer to the Button_State_Manager holding the button state
t	the button just pressed

4.4 input.h

Go to the documentation of this file.

```
00001
00005
00006 #ifndef __INPUT_H__
00007 #define __INPUT_H__
00008 #include <clay.h>
00009 #include "canvas.h"
00010
00012 typedef enum {
00013     KEY_JUST_PRESSED,
00014     KEY_PRESSED,
00015     KEY_RELEASED,
00016     KEY_JUST_RELEASED
00017 } keystate;
00018
00020 typedef enum {
00021     KEY_0,
00022     KEY_1,
00023     KEY_2,
00024     KEY_3,
00025     KEY_4,
00026     KEY_5,
00027     KEY_6,
00028     KEY_7,
```

```
00029     KEY_8,
00030     KEY_9,
00031     KEY_A,
00032     KEY_B,
00033     KEY_C,
00034     KEY_D,
00035     KEY_E,
00036     KEY_F,
00037     KEY_G,
00038     KEY_H,
00039     KEY_I,
00040     KEY_J,
00041     KEY_K,
00042     KEY_L,
00043     KEY_M,
00044     KEY_N,
00045     KEY_O,
00046     KEY_P,
00047     KEY_Q,
00048     KEY_R,
00049     KEY_S,
00050     KEY_T,
00051     KEY_U,
00052     KEY_V,
00053     KEY_W,
00054     KEY_X,
00055     KEY_Y,
00056     KEY_Z,
00057     KEY_MINUS,
00058     KEY_PLUS,
00059     MOUSE_BUTTON_LEFT,
00060     MOUSE_BUTTON_RIGHT,
00061     MOUSE_BUTTON_MIDDLE,
00062     NUM_KEYS
00063 } keys;
00064
00066 typedef struct {
00067     keystate key_status[NUM_KEYS];
00068     double mouse_x;
00069     double mouse_y;
00070     double prev_x;
00071     double prev_y;
00072     double scroll_x;
00073     double scroll_y;
00074 } Input_Handler;
00075
00077 typedef enum {
00078     BUTTON_DRAW,
00079     BUTTON_ERASE,
00080     BUTTON_SHAPE,
00081     BUTTON_FILE,
00082     BUTTON_LINE,
00083     BUTTON_EMPTY_RECT,
00084     BUTTON_FILLED_RECT,
00085     BUTTON_EMPTY_CIRCLE,
00086     BUTTON_FILLED_CIRCLE,
00087     BUTTON_NONE,
00088 } Button_Type;
00089
00091 typedef struct {
00092     // Button_State buttons[NUM_BUTTONS];
00093     Button_Type stb;
00094     Button_Type smb;
00095     Clay_String sbp;
00096 } Button_State_Manager;
00097
00104 void input_handler_init(Input_Handler *ih);
00109 void update_key_state(Input_Handler *ih);
00119 int cursor_in_bounds(Input_Handler *ih, float min_x, float min_y, float max_x, float max_y);
00120
00125 void button_state_manager_init(Button_State_Manager *bs);
00132 void process_input(Input_Handler *ih, Button_State_Manager *bs_manager, Canvas *c);
00139 void update_sbp(Button_State_Manager *bs_manager, Button_Type t);
00140
00141 #endif
```

4.5 src/math_utils.h File Reference

Contains the definition various mathematical helper functions/values.

```
#include <stdint.h>
```

4.5.1 Detailed Description

Contains the definition various mathematical helper functions/values.

4.6 math_utils.h

Go to the documentation of this file.

```
00001
00005 #ifndef MATH_UTILS_H
00006 #define MATH_UTILS_H
00007
00008 #include <stdint.h>
00009
00017 static inline int32_t clampi(int32_t val, int32_t min, int32_t max) {
00018     if(val < min) return min;
00019     if(val > max) return max;
00020     return val;
00021 }
00029 static inline uint32_t clampui(uint32_t val, uint32_t min, uint32_t max) {
00030     if(val < min) return min;
00031     if(val > max) return max;
00032     return val;
00033 }
00034
00038 static inline int32_t maxi(int32_t a, int32_t b) {
00039     return (a > b) ? a : b;
00040 }
00044 static inline uint32_t maxui(uint32_t a, uint32_t b) {
00045     return (a > b) ? a : b;
00046 }
00050 static inline int32_t mini(int32_t a, int32_t b) {
00051     return (a < b) ? a : b;
00052 }
00056 static inline uint32_t minui(uint32_t a, uint32_t b) {
00057     return (a < b) ? a : b;
00058 }
00059
00060 #endif // MATH_UTILS_H
```

4.7 src/ui_core.h File Reference

Contains structs/functions for UI using Clay.

```
#include <bob.h>
#include <clay.h>
#include <stdint.h>
#include "canvas.h"
#include "input.h"
```

Data Structures

- struct Custom_Element_Data

  Struct used to hold the data of a Clay custom element as a tagged union.

Enumerations

- enum Custom_Element_Type { **CUSTOM_ELEMENT_TYPE_CANVAS** }

  Enum used to tag different Clay Custom element types.
- enum Button_State { **BUTTON_UP** , **BUTTON_DOWN** }

  Enum to hold the different states a button could be in.

Functions

- void create_static_button (uint32_t id, uint32_t width, uint32_t height, uint8_t padding, Clay_Color hover_↩ colour, Clay_Color basic_colour, Clay_String text, void(∗button_handler)(Clay_ElementId id, Clay_Pointer↩ Data info, void ∗userData), void ∗userData)

  Initialises a Button_State_Manager by setting each button to have state BUTTON_UP.
- void clay_init (size_t width, size_t height)

  Initialises Clay.
- void clay_update_dimensions (Input_Handler ∗ih, float dt, size_t width, size_t height)

  Updates the screen dimensions used by clay as well as mouse/scroll positions.
- void clay_render (BOB_Renderer_Handle r, Clay_RenderCommandArray renderCommands)

  Draws the Clay UI to the screen.
- void **clay_free** ()

  Destroys any resources used by clay during rendering.
- Clay_RenderCommandArray editor_layout (float dt, Custom_Element_Data ∗canvas)

  Defines the Clay layout for the main editor page.

Variables

- BOB_Font_Handle **bitmap**

  Global currently used to store the bitmap font used for text rendering.
- Button_State_Manager **bs_manager**

4.7.1 Detailed Description

Contains structs/functions for UI using Clay.

4.7.2 Function Documentation

4.7.2.1 clay_init()

```
void clay_init (
            size_t width,
            size_t height)
```

Initialises Clay.

Parameters

width	the width of the region the Clay UI should occupy
height	the height of the region the Clay UI should occupy

4.7.2.2 clay_render()

```
void clay_render (
            BOB_Renderer_Handle r,
            Clay_RenderCommandArray renderCommands)
```

Draws the Clay UI to the screen.

Parameters

r	the handle of the BOB_Renderer used for drawing
renderCommands	an array containing all of the necessary draw commands

4.7.2.3 clay_update_dimensions()

```
void clay_update_dimensions (
            Input_Handler * ih,
            float dt,
            size_t width,
            size_t height)
```

Updates the screen dimensions used by clay as well as mouse/scroll positions.

Parameters

ih	a pointer to the input handler used to hold the new mouse/scroll positions
dt	the time elapsed since last frame (in milliseconds)
width	the new region width
height	the new region height

4.7.2.4 create_static_button()

```
void create_static_button (
            uint32_t id,
            uint32_t width,
            uint32_t height,
            uint8_t padding,
            Clay_Color hover_colour,
            Clay_Color basic_colour,
            Clay_String text,
            void(* button_handler )(Clay_ElementId id, Clay_PointerData info, void *userData),
            void * userData)
```

Initialises a Button_State_Manager by setting each button to have state BUTTON_UP.

Parameters

bs	a pointer to the Button_State_Manager to initialise Creates a clay button widget whose size does not change

id	the unique numeric id of the button to differentiate it from other buttons
width	the width of the button
height	the height of the button
padding	the padding of the button
hover_colour	the colour the button should turn when the mouse hovers over it
basic_colour	the colour the button should stay when not hovered over
text	the text the button should contain
button_handler	callback function that should be called when the button is clicked
userData	the data that should be passed to the button_handler callback

4.7.2.5 editor_layout()

```
Clay_RenderCommandArray editor_layout (
            float dt,
            Custom_Element_Data * canvas)
```

Defines the Clay layout for the main editor page.

Parameters

dt	the time elapsed since last frame (in milliseconds)
canvas	a pointer to the Custom_Element_Data holding the render data for the canvas

Returns

a Clay_RenderCommandArray that holds all the draw commands for this layout

4.8 ui_core.h

Go to the documentation of this file.

```
00001
00005 #ifndef UI_CORE_H
00006 #define UI_CORE_H
00007
00008 #include <bob.h>
00009 #include <clay.h>
00010 #include <stdint.h>
00011 #include "canvas.h"
00012 #include "input.h"
00013
00015 extern BOB_Font_Handle bitmap; //TODO: Replace this with something else so we can use different font
      sizes for different widgets
00016
00018 typedef enum {
00019     CUSTOM_ELEMENT_TYPE_CANVAS,
00020 } Custom_Element_Type;
00021
00022 extern Button_State_Manager bs_manager; //> Manages UI bs
00023
00027 typedef struct {
00028     Custom_Element_Type type;
00029     union {
00030         Canvas *canvas;
00031     };
00032 } Custom_Element_Data;
00033
```

```
00035 typedef enum {
00036     BUTTON_UP,
00037     BUTTON_DOWN,
00038 } Button_State;
00039
00044 // void button_state_manager_init(Button_State_Manager *bs);
00045 // void update_sbp(Button_Type t);
00046
00059 void create_static_button(uint32_t id, uint32_t width, uint32_t height, uint8_t padding, Clay_Color
      hover_colour,
00060                           Clay_Color basic_colour, Clay_String text, void
      (*button_handler)(Clay_ElementId id,
00061                           Clay_PointerData info, void *userData), void *userData);
00062
00068 void clay_init(size_t width, size_t height);
00076 void clay_update_dimensions(Input_Handler *ih, float dt, size_t width, size_t height);
00082 void clay_render(BOB_Renderer_Handle r, Clay_RenderCommandArray renderCommands);
00086 void clay_free();
00093 Clay_RenderCommandArray editor_layout(float dt, Custom_Element_Data *canvas);
00094
00095 #endif // UI_CORE_H
```

Index

BUTTON_DRAW
input.h, 18
BUTTON_EMPTY_CIRCLE
input.h, 18
BUTTON_EMPTY_RECT
input.h, 18
BUTTON_ERASE
input.h, 18
BUTTON_FILE
input.h, 18
BUTTON_FILLED_CIRCLE
input.h, 18
BUTTON_FILLED_RECT
input.h, 18
BUTTON_LINE
input.h, 18
BUTTON_NONE
input.h, 18
BUTTON_SHAPE
input.h, 18
Button_State_Manager, 5
button_state_manager_init
input.h, 18
Button_Type
input.h, 17

Canvas, 6
canvas.h
canvas_begin_shape, 10
canvas_bresenham, 10
canvas_draw, 11
canvas_draw_empty_circle, 11
canvas_draw_empty_rect, 11
canvas_draw_filled_circle, 12
canvas_draw_filled_rect, 12
canvas_draw_line, 13
canvas_dump_set, 13
canvas_end_shape, 14
canvas_free, 14
canvas_init, 14
canvas_print, 15
canvas_set_pixel, 15
canvas_begin_shape
canvas.h, 10
canvas_bresenham
canvas.h, 10
canvas_draw
canvas.h, 11
canvas_draw_empty_circle
canvas.h, 11
canvas_draw_empty_rect
canvas.h, 11
canvas_draw_filled_circle
canvas.h, 12
canvas_draw_filled_rect
canvas.h, 12
canvas_draw_line
canvas.h, 13
canvas_dump_set
canvas.h, 13
canvas_end_shape
canvas.h, 14
canvas_free
canvas.h, 14
canvas_init
canvas.h, 14
canvas_print
canvas.h, 15
canvas_set_pixel
canvas.h, 15
clay_init
ui_core.h, 23
clay_render
ui_core.h, 23
clay_update_dimensions
ui_core.h, 24
create_static_button
ui_core.h, 24
cursor_in_bounds
input.h, 18
Custom_Element_Data, 6
Custom_Element_Data::[union].__unnamed0__, 5

editor_layout
ui_core.h, 25

input.h
BUTTON_DRAW, 18
BUTTON_EMPTY_CIRCLE, 18
BUTTON_EMPTY_RECT, 18
BUTTON_ERASE, 18
BUTTON_FILE, 18
BUTTON_FILLED_CIRCLE, 18
BUTTON_FILLED_RECT, 18
BUTTON_LINE, 18
BUTTON_NONE, 18
BUTTON_SHAPE, 18
button_state_manager_init, 18
Button_Type, 17
cursor_in_bounds, 18

input_handler_init, 19
KEY_JUST_PRESSED, 18
KEY_JUST_RELEASED, 18
KEY_PRESSED, 18
KEY_RELEASED, 18
keystate, 18
process_input, 19
update_key_state, 20
update_sbp, 20
Input_Handler, 7
input_handler_init
input.h, 19

KEY_JUST_PRESSED
input.h, 18
KEY_JUST_RELEASED
input.h, 18
KEY_PRESSED
input.h, 18
KEY_RELEASED
input.h, 18
keystate
input.h, 18

process_input
input.h, 19

src/canvas.h, 9, 16
src/input.h, 16, 20
src/math_utils.h, 21, 22
src/ui_core.h, 22, 25

ui_core.h
clay_init, 23
clay_render, 23
clay_update_dimensions, 24
create_static_button, 24
editor_layout, 25
update_key_state
input.h, 20
update_sbp
input.h, 20